Optimizers

Batch GD, SGD, Momentum and Adam

The problem

- Minimise a loss L(w) over many parameters
- Full gradients are expensive on big data
- Ravines and noise slow plain descent

Batch gradient descent

$$w \leftarrow w - \eta \nabla L(w)$$

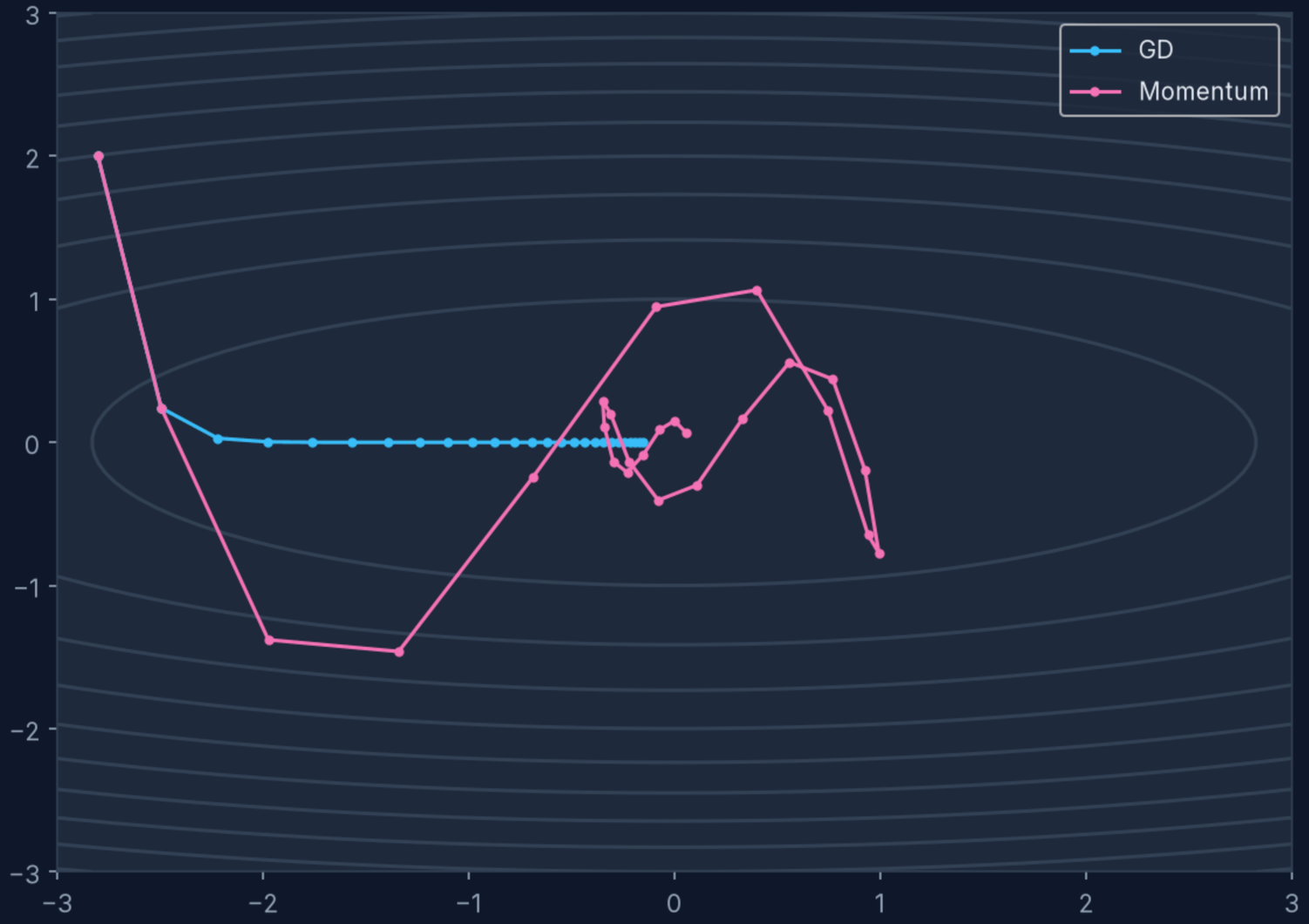
- Exact gradient over all n examples
- Learning rate η: too big diverges, too small crawls

Batch GD and mini-batch SGD

```python
def batch_gd(grad, w0, X, y, lr=0.1, epochs=100):
    w = np.array(w0, float)
    for _ in range(epochs):
        w -= lr * grad(w, X, y)
    return w

def minibatch_sgd(grad, w0, X, y, lr=0.1, epochs=50, batch_size=16, seed=0):
    rng = np.random.default_rng(seed)
    w = np.array(w0, float)
    n = len(X)
    for _ in range(epochs):
        idx = rng.permutation(n)
        for s in range(0, n, batch_size):
            b = idx[s:s + batch_size]
            w -= lr * grad(w, X[b], y[b])
    return w
```

Momentum



$$v \leftarrow \beta v + \nabla L$$

$$w \leftarrow w - \eta v$$

Damps zig-zags

Accelerates along valleys

Adam

$$m \leftarrow \beta_1 m + (1-\beta_1) g, \quad v \leftarrow \beta_2 v + (1-\beta_2) g^2$$

$$w \leftarrow w - \eta \frac{\hat{m}}{\sqrt{\hat{v}} + \epsilon}$$

- Per-parameter step sizes
- Bias correction: $\hat{m} = m / (1 - \beta_1^t)$

Adam

```python
def adam(grad, w0, X, y, lr=0.05, b1=0.9, b2=0.999, eps=1e-8, epochs=300):
    w = np.array(w0, float)
    m, v = np.zeros_like(w), np.zeros_like(w)
    for t in range(1, epochs + 1):
        g = grad(w, X, y)
        m = b1 * m + (1 - b1) * g
        v = b2 * v + (1 - b2) * g * g
        mh, vh = m / (1 - b1 ** t), v / (1 - b2 ** t)
        w -= lr * mh / (np.sqrt(vh) + eps)
    return w
```

Trade-offs

- Batch GD: stable, slow per step
- SGD: cheap steps, noisy; great generalisation
- Momentum: faster through ravines
- Adam: robust default, little tuning

Recap

- Step against the gradient
- Mini-batches for scale
- Momentum smooths direction
- Adam adapts step size per parameter